

A1)

Predict what Racket will print in response to each of these expressions. Then try it and make sure your answer was correct, or if not, that you understand why!

```
(define a 3)

(define b (+ a 1))

(+ a b (* a b))

(= a b)

(if (and (> b a) (< b (* a b)))
    b
    a)

/

(-4)

(- 4)

-4

- 4
```

```
(cond ((= a 4) 6)
      ((= b 4) (+ 6 7 a))
      (else 25))

(+ 2 (if (> b a) b a))

(* (cond ((> a b) a)
        ((< a b) b)
        (else -1))
   (+ a 1))

((if (< a b) + -) a b)
```

A2)

Define a procedure `sign` that takes a number as its argument and returns 1 if the number is *positive*, -1 if the number is *negative*, and 0 if the number is *zero*.

A3)

Define a procedure `sumSquareBigger` that takes three numbers as arguments and returns the sum of the squares of the two bigger numbers.

A4)

a) Implement a procedure `areaRect` that calculates the area of a rectangle. The procedure takes 2 arguments called `length` and `width`. E.g., `(areaRect 3 4)` shall return 12.

b) Likewise, implement a procedure `circumferenceCircle` that takes the radius of a circle and calculates its circumference. Make use of the *built-in* constant `pi`.

A5)

a) Implement a procedure `mynot` that behaves like the built-in `not` (without however using the *built-in* `not` operation). E.g., `(mynot (> 3 4))` shall return `#t`.

b) In contrast to `not`, and as well as `or` cannot be implemented as procedures. Racket implements them as *special-forms*, in order to achieve (so-called) *short-circuit-evaluation*.

Give examples that demonstrates the *short-circuit-evaluation* for `and`/`or`.